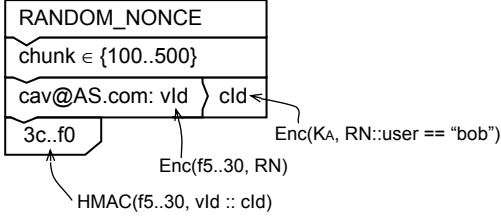


The above macaroon from the example in Figure 3 permits access to a range of chunks. By deriving a macaroon with an added third-party caveat, that authority can be delegated to a user “bob” at an authentication service, as below:



A third-party caveat message contains an root key and assertion encrypted for the discharging service (above, cld), a hint to its location (above, AS.com), and the same key encrypted for the target service to allow verification (above, vld). The signature of a discharge macaroon (below) is chained from this root key.

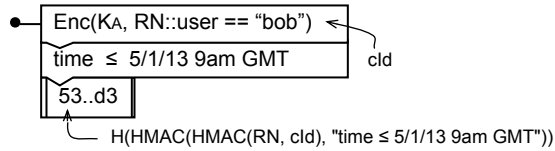


Fig. 5. A macaroon with a third-party caveat (in the middle), derived from another macaroon (at the top). At the bottom is a discharge macaroon for the caveat (whose signature is hashed to prevent further chaining), created by a third-party holder of the shared key KA after checking the assertion in the key identifier cld. By deriving the chain of nested HMAC values, the target service can verify the integrity of such discharge macaroons by decrypting the encrypted root key identifier vld.

derive new, further attenuated macaroons. In this manner, macaroon signatures both protect integrity and act as keys.

The third-party caveats in macaroons are requirements for holder-of-key proofs: assertions of one or more predicates signed by the holder of a key. The manner in which these requirements are embedded into a macaroon, and its chain of HMAC values, allows (i) the appropriate parties to create proofs asserting those predicates, (ii) those proofs, themselves, to embed additional caveats (even for new, additional third parties), and, (iii) the target service to verify that all required proofs have been discharged, recursively.

A proof that discharges a macaroon’s third-party caveat is itself a macaroon, as shown in Figure 5 (at the bottom). Such caveat discharge macaroons have a key identifier that encodes both the root key for deriving holder-of-key proofs and the predicates to be asserted by the third party (in Figure 5, it is denoted cld, and contains the key RN paired with the caveat predicate user == “bob” encrypted under the key KA). Conceptually, this key identifier can be thought of as a message encrypted with a public key for the third-party.

This same caveat key identifier is also contained in the message that a macaroon embeds for a third-party caveat, along with a hint to the location of caveat-discharging service (in Figure 5, that hint is @AS.com). This message also contains another, encrypted copy of the root key for holder-of-key proofs, encrypted with a key from the signature of

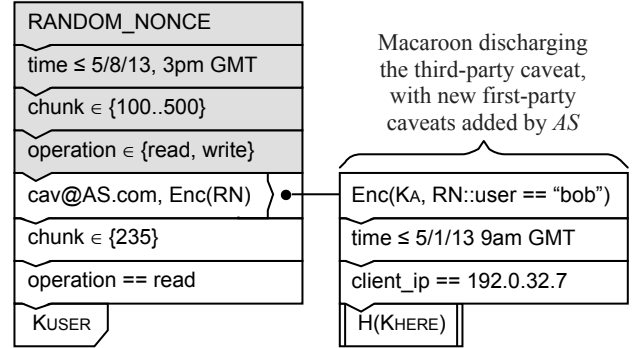
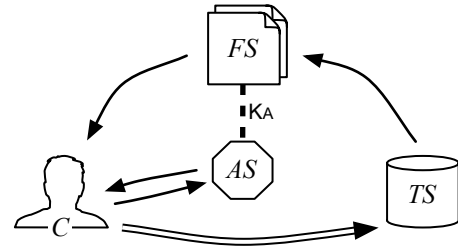


Fig. 6. Two macaroons that together give client C limited-time read access to chunk 235 at the key-value storage service TS. Using its macaroon from TS, the “forum service” FS derives for C a macaroon (on the left) that embeds a third-party caveat for an authentication service AS, confining access to clients C where a user “bob” is logged into AS. That caveat contains an encrypted root key and a key identifier. Using that key identifier, AS derives for C a macaroon (on the right) that asserts, for only a limited time, and for only one IP address, that AS has verified client C’s user.

the macaroon onto which the third-party caveat is added (in Figure 5, it is denoted vld, and is the root key RN, encrypted using the HMAC value f5..30). The target server can decrypt this second copy of the root key, and use it to verify the signatures of caveat discharge macaroons.

In general, to authorize a request, a target service may need to verify an entire set of macaroons presented with that request, and establish (i) that caveat discharge macaroons can be found for all third-party caveats, and (ii) that all first-party caveat predicates hold true in the context of the request. Target services can perform this verification by inducing a tree from the set of presented macaroons, and checking their macaroon signatures against the chain of HMAC values derived along each branch from the root to the leaves. To build this tree, the messages in the third-party caveats allow the target service to match the key identifiers of caveat discharge macaroons and decrypt their root keys—in Figure 5, respectively, the identifier Enc(KA, RN::user == “bob”) and root key Enc(f5..30, RN).

The main advantages and innovative aspects of macaroons are illustrated by the example of Figure 6, in combination. From its existing macaroon for the service TS, the forum service FS can derive a macaroon for user “bob” that attenuates *what* aspects of TS are accessible; simultaneously, FS can delegate the authentication of users to a third-party service AS, using the key KA it holds for AS. (Of course, for this, FS must know AS, and trust it to perform authentication.) The AS service can evaluate the request from client C however it chooses, before returning a macaroon that discharges the user == “bob” caveat. Notably, this caveat discharge macaroon may be very short lived (e.g., be valid for minutes, or seconds), due to the efficiency and low overhead macaroons’ HMAC-based